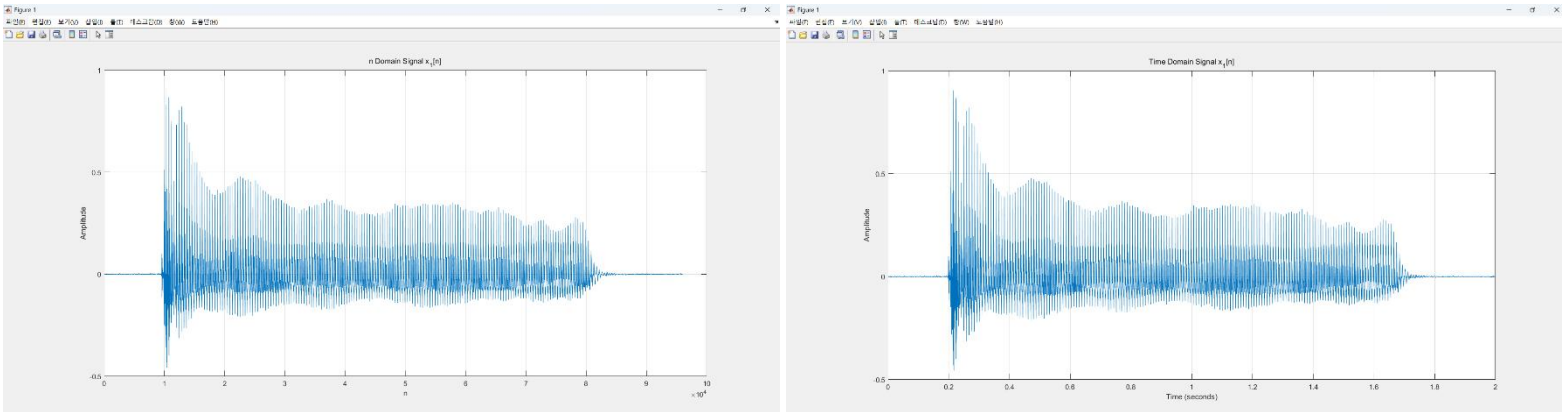


디지털 신호 처리 (EEE4420.01-00) – Simulation1 과제 1

1. discrete 한 음성신호 $x_1[n]$

$x_1[n]$ 특징: (96,000 point (2s), Sampling rate $f_s = 48,000\text{Hz}$, $T_s = \frac{1}{48,000}\text{sec}$, $\omega_s = 96,000\pi$)

1-1. Time domain에서 $x_1[n]$ 의 파형



(n-domain에서 $x_1[n]$ 의 파형 결과)

(time-domain으로 x축 변환한 $x_1[n]$ 의 파형 결과)

왼쪽 사진은 n-domain에서 $x_1[n]$ 신호의 파형을 보여준다. 그리고, 오른쪽 사진은 n(샘플 수)에 T_s 를 곱하여 n-domain을 T-domain으로 변환한 결과이다.

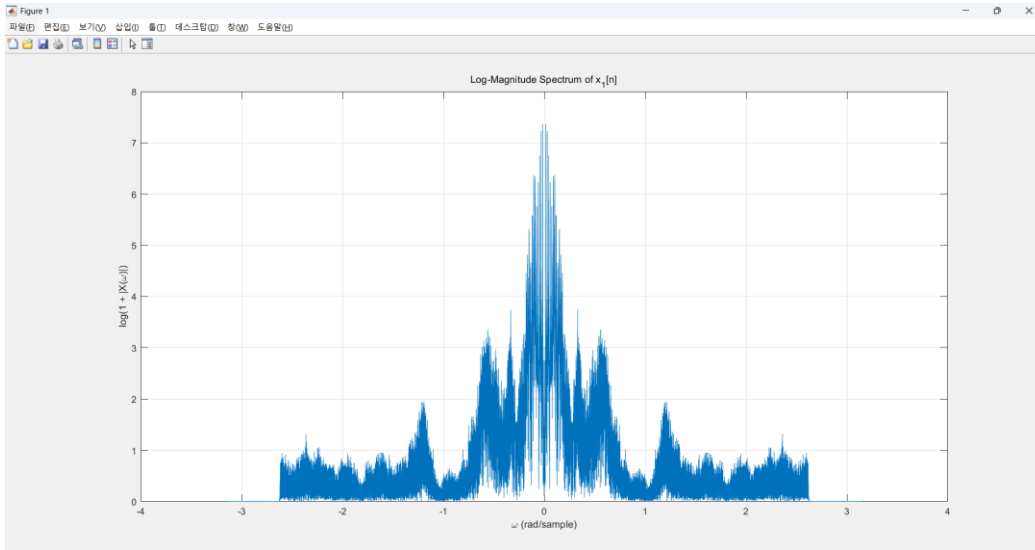
위 파형을 통하여 알 수 있는 것은 다음과 같다.

- 1) 현재 이 신호는 초반에 강한 고주파 신호를 출력하다가 0.2초 정도 후부터는, 적당한 크기 및 저주파를 유지하는 신호이다.
- 2) 이 신호는 0.2초정도 후(샘플 index $n \approx 10000$)에 강하게 출력이 되고, 바로 적당히 약해져 1.7초까지(샘플 index $n \approx 82000$) 출력되다가, 그 이후로는 매우 약해져 크기가 0에 가까워진다.
- 3) 처음에는 강하게 출력되고 이후로 일정하게 유지되는 것으로 보아, 사람의 발성이라는 가정을 하였을 때, 호흡을 참았다가 한 번에 내보내면서 나는 소리라고 추측할 수 있다.

소리를 청취한 결과

- 1) 소리를 청취한 결과 이 파형은 사람이 “아”소리를 내는 음성이었다.
- 2) 초반의 강한 고주파가 보인 이유는 녹음을 위하여 준비 후 시작하면서 바로 약간 높은 음의 “아” 소리를 내었기 때문이라고 생각된다.
- 3) 초반을 제외한 구간에서 일정한 크기와 주파수를 가지는데, 이는 초반 이후에 안정되게 “아”소리를 일정하게 내었기 때문이었다.

1-2. $x_1[n]$ 의 frequency domain Spectrum $X(\omega)$ 의 Magnitude 그래프 (Log scale)



먼저, DTFT와 DFT의 수식에 대해서 살펴본다.

DTFT는 다음과 같다.

$$X(\omega) = \sum_{n=-\infty}^{\infty} x[n]e^{-j\omega n} \quad x[n] = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(\omega)e^{j\omega n} d\omega$$

N-point DFT는 다음과 같다.

$$X(k) = \sum_{n=0}^{N-1} x[n]e^{-j\frac{2\pi}{N}kn} \quad x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X(k)e^{j\frac{2\pi}{N}kn} \quad \text{where, } \omega = \frac{2\pi}{N}k$$

이와 같은 차이점은, computer는 정보를 Memory cell안에 저장하기 때문에, 모든 것을 discrete한 디지털 신호로 처리해야 하기 때문에 나타난다.

수학적으로 DTFT의 경우에 ω -domain의 $X(\omega)$ 는 $M-1 \rightarrow \infty$ 로 보내기 때문에, 무한대의 해상도를 가지고, 파형이 연속적으로 나타난다. 하지만, DFT의 경우 $M-1 \rightarrow \infty$ 가 불가능하기에 이를 N 으로 limit시켜 Fourier Transform을 수행한다.

DFT에서 N 은 샘플의 수를 의미하며, $\omega = \frac{2\pi}{N}k$, $\omega_0 = \frac{2\pi}{N}$ 로 ω 의 해상도가 제한된다. 따라서, DFT를 하게 되면, Fourier Transform한 결과가 Continuous가 아닌, Discrete하게 나온다.

참고로, $x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X(k)e^{j\frac{2\pi}{N}kn}$ 에서 $\frac{1}{2\pi}$ 가 아니라 $\frac{1}{N}$ 이 곱해지는 이유는 DFT에서는 $X(\omega)$ 가 아닌 $X(k)$ 를 사용하기 때문이다.

위 파형을 통하여 알 수 있는 것은 다음과 같다.

- 1) 저주파에 많은 정보들이 몰려 있고, 저주파의 신호의 세기가 다른 주파수 대역보다 강하다.
- 2) $-\pi \sim \pi$ 까지 전부 $X(k)$ 가 존재하지 않고, 약 $\frac{5\pi}{6} \approx 2.6180$ 까지만 존재하기에, LPF를 한번 거친 신호임을 추측할 수 있다.

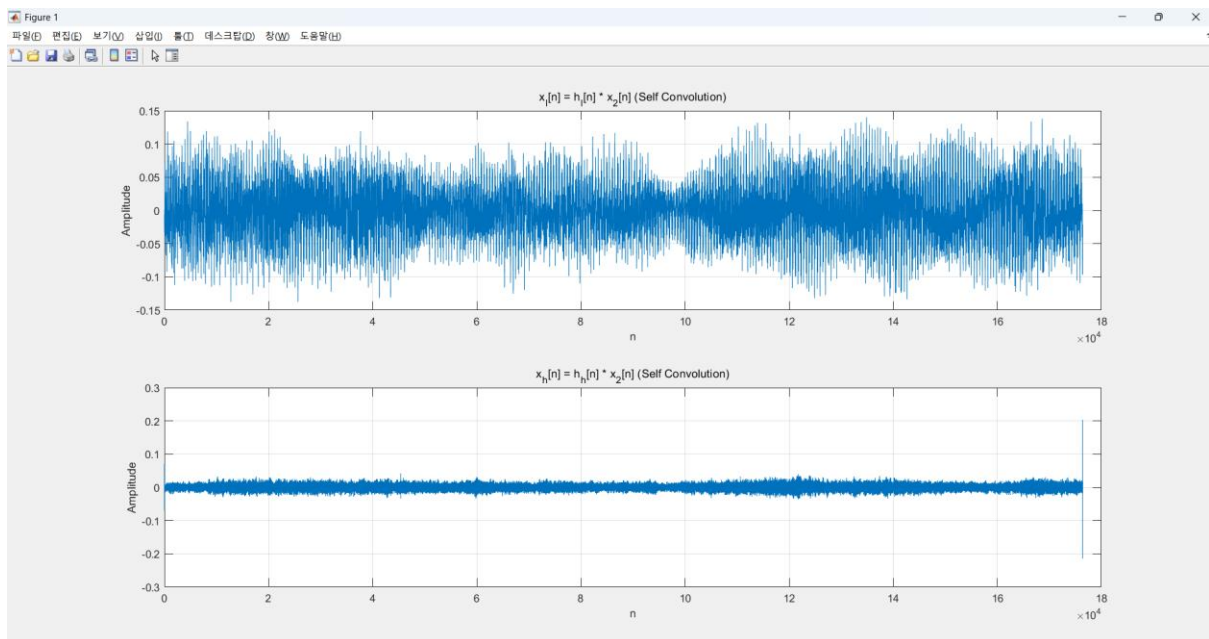
3) Magnitude를 log scale로 표시한 이유는 주파수 성분의 크기가 매우 넓은 범위에 걸쳐 있기 때문이다. 같은 주파수를 가지는 신호의 세기가 모두 더해지므로, 세기의 편차가 심하다.

2. 2개의 impulse response $h_h[n]$ (HPF.txt), $h_l[n]$ (LPF.txt)를 사용하여,
주어진 음성신호 $x_2[n]$ 을 time domain에서 filtering 하는 과정

$$\begin{cases} x_l[n] = h_l[n] * x_2[n] \\ x_h[n] = h_h[n] * x_2[n] \end{cases} \quad (1)$$

(176,400 point (4s), Sampling rate $f_s = 44,100\text{Hz}$, $T_s = \frac{1}{44,100}\text{sec}$, $\omega_s = 88,200\pi$)

2-1. discrete convolution을 직접 구현 및 이를 사용하여 구한 $x_l[n]$ 와 $x_h[n]$ 의 파형



먼저, convolution의 수식을 살펴본다.

$$y[n] = \sum_{m=-\infty}^{\infty} x[m] h[n-m]$$

위 수식을 직접 코드로 구현하려면, 이중 for문을 사용한다.

먼저, conv된 신호의 길이는 x신호의 길이 + h신호의 길이 - 1이다.

이를 $N_y = N_x + N_h - 1$ 이라고 하겠다.

바깥쪽 for문은 $y[n]$ 의 n 번째 index를 지정해주기 위해 n 으로 잡고, 범위는 1 ~ N_y 까지로 한다.

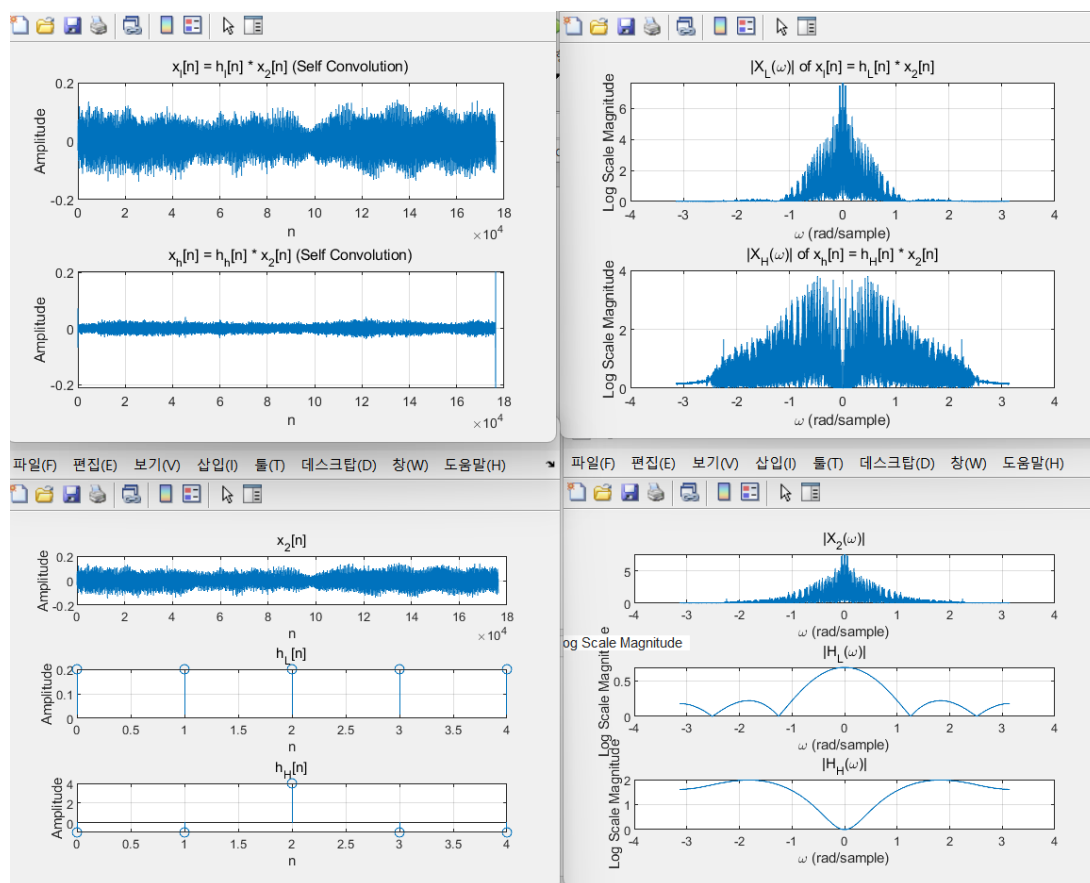
안쪽 for문은 남은 m 으로 iteration을 돌려주고, $x[m]$ 과 $h[n-m]$ 이 겹치는 부분을 곱하여 더한다.

위 사진은 convolution결과의 n-domain파형을 보여준다. 위쪽은 $x_2[n]$ 과 LPF의 convolution, 아래쪽은 $x_2[n]$ 과 HPF의 convolution이다.

위 파형을 통하여 알 수 있는 것은 다음과 같다.

- 1) $x_2[n]$ 신호와 LPF와 convolution되었을 때는 신호의 amplitude가 강하다. 이는, $x_2[n]$ 신호는 저주파 성분의 세기가 크다는 것을 의미한다. 반면, HPF와 convolution되었을 때는 amplitude가 약한데, 이는 고주파 성분의 세기가 약하다는 것을 의미한다.
- 2) $x_2[n]$ 신호와 LPF와 convolution된 신호가 HPF와 convolution된 신호보다 더 많은 정보를 가지고 있다는 것을 알 수 있고, 이를 통해 LPF와 convolution은 핵심적인 information은 유지한다는 것을 알 수 있다.
- 3) HPF와 convolution된 신호는 많은 정보들이 사라졌으므로, 잡음이 많이 들리거나 원래 음성의 고주파 부분만 남았을 것으로 추측된다.

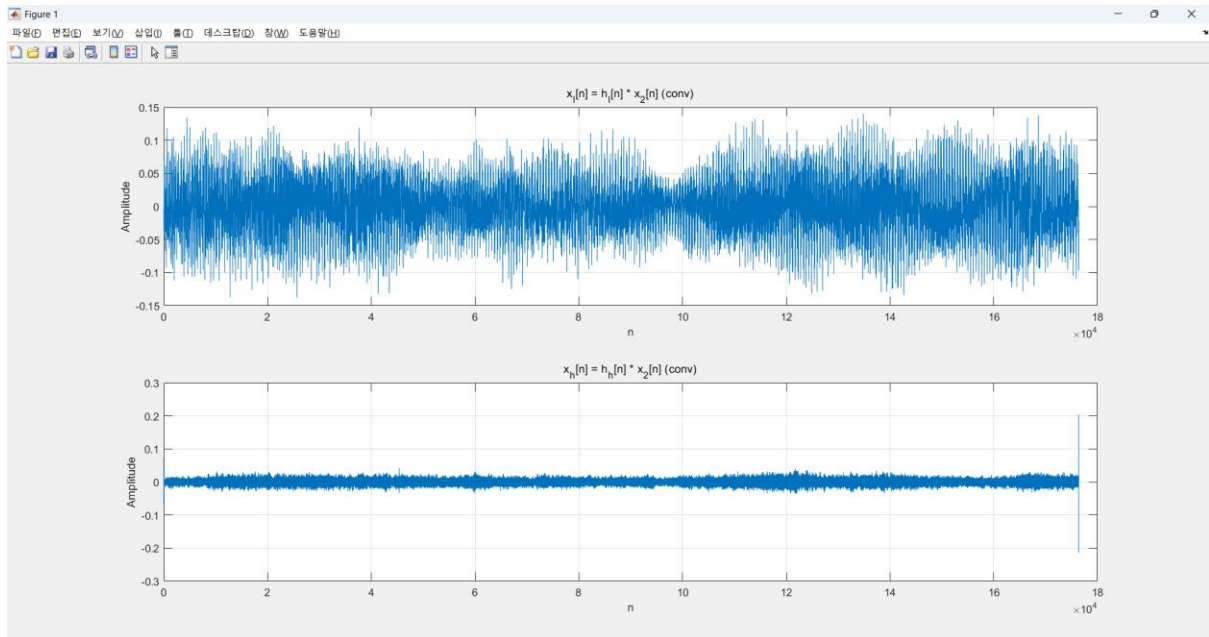
추가 파형 plot을 통한 분석



Convolution Property에 의해 $n - domain$ 에서 convolution은 $\omega - domain$ 에서 multiplication이다.

$\omega - domain$ 을 보면, LPF에서는 $X_2(\omega)$ 의 저주파 성분이 주로 남았고, HPF에서는 $X_2(\omega)$ 의 저주파 성분이 사라진 것을 확인할 수 있다. 더불어, $X_H(\omega)$ 의 magnitude가 $X_2(\omega)$ 의 magnitude에 비해 매우 작기에 핵심적인 Low Frequency쪽의 정보들이 사라진 것을 확인할 수 있다.

2-2. Matlab 내장함수를 사용한 $x_l[n]$ 와 $x_h[n]$ 의 파형

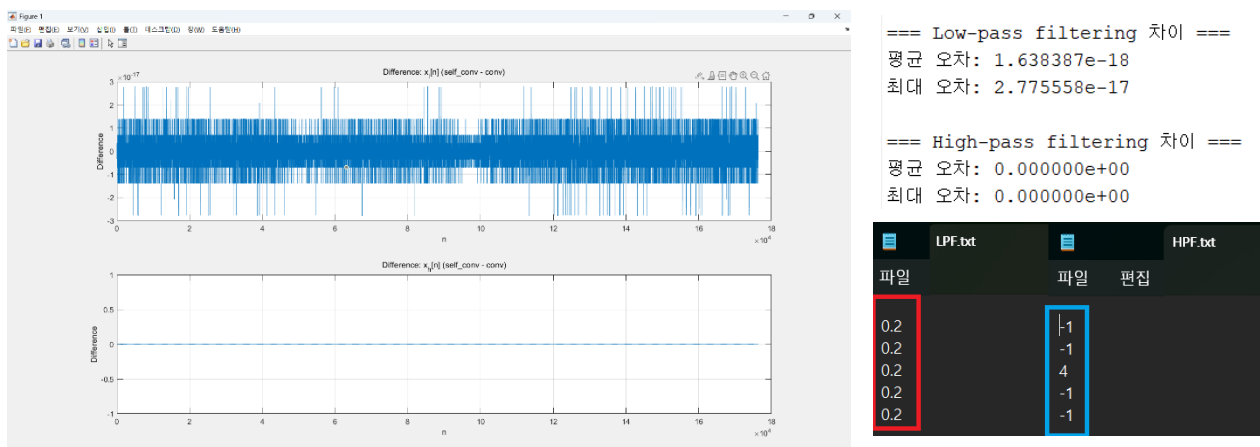


위 파형은 Matlab 내장함수인 conv()를 이용하여, 2-1과 똑같은 과정을 진행한 것이다.

육안으로 보면, 결과가 똑같이 나오는 것을 확인할 수 있다.

하지만, 세밀한 분석을 위하여, 직접 구현한 conv와 내장함수 conv를 이용하여, 출력된 결과들을 빼서 어느 정도 오차가 있는지 확인하여 보았다.

2-1의 결과와 일치하는지 확인



위 차이를 plot한 결과 및, 오차를 측정한 결과를 보면, LPF에서 미세한 차이가 나는 것을 확인할 수 있다. 이 오차는 매우 미세하기에 아마도 float 변수 계산 알고리즘에 차이가 있어서 발생한 것으로 추측된다. txt파일을 확인해보면, LPF는 0.2로 float형인 것을 확인할 수 있다. 오차가 매우 미세하기에 실질적인 성능은 직접 구현한 convolution과 내장함수 conv()가 같다고 할 수 있을 것 같다.

2-3. $x_l[n]$ 와 $x_h[n]$ 을 각각 청취한 뒤, 원본 신호 $x_2[n]$ 과 어떤 차이가 있는지 기술

원본 음성 청취

원본 음성 신호는 전형적인 클래식 음악의 소리를 포함하고 있다.

$x_l[n]$ 음성 청취

LPF가 적용된 신호는, 고주파 성분이 제거되고, 저주파 성분만 남는다. 이로 인해 음성을 청취하였을 때, 음성은 부드럽고 둔탁한 느낌을 주었다. 즉, 음악의 저음부분이 부각되어 들리지만, 고음이 부족하여 음악이 덜 이쁘게 들렸고 섬세한 음향이 사라진 느낌을 받았다. 위와 같은 차이점이 있긴 했지만, LPF와 원본신호의 차이는 미세한 차이였고, LPF가 적용된 신호는 거의 원본 신호와 비슷했다.

$x_h[n]$ 음성 청취

HPF가 적용된 신호는 저주파 성분이 제거되고 고주파 성분만 남는다. 이로 인해 음성을 청취하였을 때 날카롭고 얇은 소리를 들을 수 있었고, 틱틱소리와 같은 잡음도 들을 수 있었다. 즉, 저음이 사라져 고음이 강조된 결과, 저음이 부족하여 음악이 풍부하지 않고 부족한 느낌이 들었고, 소리가 매우 약하고 건조한 느낌을 주었다.

정리하면, 원본 신호는 저주파와 고주파 성분이 혼합되어 있어, 풍부한 음향을 느낄 수 있었다. 하지만, LPF를 적용한 신호는 저음부만 강조되고, 전체적인 흐름만 남은 느낌이었고, HPF가 적용된 신호는 얇은 소리 및 노이즈가 들리고, 신호의 세기가 약해진 느낌이 들었다.

더불어, 직접 구현한 convolution과 내장함수를 이용한 결과 신호를 비교하였을 때, 들리는 음성으로는 차이가 느껴지지 않았다.

3. Appendix

```
% 1-1. x1[n]을 n-domain에서 plot & sound 듣기 (x축 = n-domain)

% clear;
% clc;
% close all;
%
% % x1.wav 파일 불러와 읽고, Fs(Sampling Rate)를 가져온다.
% [x1, Fs] = audioread('x1.wav'); % Fs = 48000 (Hz)
% N = length(x1); % 96000 point (총 샘플 수)
%
% % n-domain에 plot해야 하므로, n의 범위를 정해준다.
% n = 0:N-1;
%
% % n-domain에서 x1[n]을 plot
% figure;
% plot(n, x1);
% xlabel('n');
% ylabel('Amplitude');
% title('n Domain Signal x_1[n]');
% grid on;
%
% % sound 재생
% sound(x1, Fs);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1-1. x1[n]을 t-domain에서 plot & sound 듣기 (x축 = t-domain)

clear;
clc;
close all;

% x1.wav 파일 불러와 읽고, Fs(Sampling Rate)를 가져온다.
[x1, Fs] = audioread('x1.wav'); % Fs = 48000 (Hz)
Ts = 1 / Fs; % Ts는 샘플링 주기 (F=1/T)
N = length(x1); % 96000 point (총 샘플 수)

% t-domain의 범위를 정해준다.
% 샘플 수에 샘플링 주기를 곱하면, 몇 sec동안 음악이 나오는지 알 수 있다.
% x축을 통해 몇 sec동안 파형이 지속되는지 표기하는 것은
% n-domain을 t-domain으로 바꾸는 것과 동일하다
t = (0:N-1) * Ts;

% t-domain에서 x1[n]을 plot
figure;
plot(t, x1);
xlabel('Time (seconds)');
ylabel('Amplitude');
title('Time Domain Signal x_1[n]');
grid on;

% sound 재생
sound(x1, Fs);
```

```

% 1-2. frequency domain에서의 Spectrum Magnitude (Log scale)

clear;
clc;
close all;

% x1.wav 파일 불러와 읽고, Fs(Sampling Rate)를 가져온다.
[x1, Fs] = audioread('x1.wav'); % Fs = 48000 Hz
N = length(x1); % 96000 point (총 샘플 수)

% FFT 계산
X = fft(x1, N); % N-point FFT (DFT 대신 사용) (N = 96000)

% Magnitude 계산 및 log scale 적용
X_mag = abs(X); % Magnitude
X_log = log(1 + X_mag); % Log scale

% fftshift로  $-\pi \sim \pi$  범위로 바꿔줌
X_log_shifted = fftshift(X_log);

%  $\omega$ -domain 범위를 정해준다. ( $\omega$ 는  $[-\pi, \pi]$ 까지)
w = linspace(-pi, pi, N);

%  $\omega$ -domain에서  $F\{x_1[n]\} = X(\omega)$ 을 plot
figure;
plot(w, X_log_shifted);
xlabel('\omega (rad/sample)');
ylabel('log(1 + |X(\omega)|)');
title('Log-Magnitude Spectrum of x_1[n]');
grid on;

```

```

% =====
% 2-1. Discrete convolution 직접 구현
% =====

clear;
clc;
close all;

%%%%%%%%%% x2.wav 및 LPF, HPF 읽기 %%%%%%%%%%
[x2, Fs] = audioread('x2.wav'); % x2[n], Fs = 44100 Hz
h1 = importdata('LPF.txt'); % LPF impulse response
hh = importdata('HPF.txt'); % HPF impulse response

%%%%%%%%%% 직접 구현한 컨볼루션 %%%%%%%%%%
x_l_custom = self_conv(x2', h1'); % LPF 적용
x_h_custom = self_conv(x2', hh'); % HPF 적용

%%%%%%%%%% n축의 범위를 정해줌 %%%%%%%%%%
n_l = 0:length(x_l_custom)-1;
n_h = 0:length(x_h_custom)-1;

%%%%%%%%%% 각각의 filter의 n-domain에서 x2[n]*h[n]을 plot %%%%%%%%%%
figure;

subplot(2, 1, 1);
plot(n_l, x_l_custom);
xlabel('n');
ylabel('Amplitude');
title('x_l[n] = h_l[n] * x_2[n] (Self Convolution)');
grid on;

subplot(2, 1, 2);
plot(n_h, x_h_custom);
xlabel('n');
ylabel('Amplitude');
title('x_h[n] = h_h[n] * x_2[n] (Self Convolution)');
grid on;

%=====
% %%%%%%%%%%% 추가: x2[n], h1[n], hh[n] n-domain plot %%%%%%%%%%%
% figure;
%
% subplot(3, 1, 1);
% n_x2 = 0:length(x2)-1;
% plot(n_x2, x2);
% xlabel('n');
% ylabel('Amplitude');
% title('x_2[n]');
% grid on;
%
% subplot(3, 1, 2);
% n_h1 = 0:length(h1)-1;
% stem(n_h1, h1);
% xlabel('n');
% ylabel('Amplitude');
% title('h_L[n]');
% grid on;
%
% subplot(3, 1, 3);
% n_hh = 0:length(hh)-1;
% stem(n_hh, hh);
% xlabel('n');
% ylabel('Amplitude');
% title('h_H[n]');
% grid on;
%
% %%%%%%%%%%% 추가: Spectrum Magnitude (Log scale) %%%%%%%%%%%
% N = length(x2);
% X2 = fft(x2, N);
% HL = fft(h1, N);
% HH = fft(hh, N);
%

```

```

% % log scale 적용
% X2_mag = abs(X2); % Magnitude
% X2_log = log(1 + X2_mag); % Log scale
% HL_mag = abs(HL);
% HL_log = log(1 + HL_mag);
% HH_mag = abs(HH);
% HH_log = log(1 + HH_mag);
%

% % fftshift로  $-\pi \sim \pi$  범위로 바꿔줌
% X2_shifted = fftshift(X2_log);
% HL_shifted = fftshift(HL_log);
% HH_shifted = fftshift(HH_log);
%
%

% %  $\omega$ -domain 범위를 정해준다. ( $\omega$ 는  $[-\pi, \pi]$ 까지)
% w = linspace(-pi, pi, N);
%
% % Plot
% figure;
%
% subplot(3, 1, 1);
% plot(w, X2_shifted);
% xlabel('\omega (rad/sample)');
% ylabel('Log Scale Magnitude');
% title('|X_2(\omega)|');
% grid on;
%
% subplot(3, 1, 2);
% plot(w, HL_shifted);
% xlabel('\omega (rad/sample)');
% ylabel('Log Scale Magnitude');
% title('|H_L(\omega)|');
% grid on;
%
% subplot(3, 1, 3);
% plot(w, HH_shifted);
% xlabel('\omega (rad/sample)');
% ylabel('Log Scale Magnitude');
% title('|H_H(\omega)|');
% grid on;
%

% %%%%%%%%%%% 추가: x_l_custom, x_h_custom Frequency Domain %%%%%%%%%%%
%
% % FFT, M-point에서 point수는 똑같이 맞춤
% N_conv = length(x_l_custom);
% X_L_custom = fft(x_l_custom, N_conv);
% X_H_custom = fft(x_h_custom, N_conv);
%
% % log scale 적용
% X_L_mag_log = log(1 + abs(X_L_custom));
% X_H_mag_log = log(1 + abs(X_H_custom));
%
% % fftshift로  $-\pi \sim \pi$  범위로 바꿔줌
% X_L_shifted = fftshift(X_L_mag_log);
% X_H_shifted = fftshift(X_H_mag_log);
%
% %  $\omega$ -domain 범위를 정해준다. ( $\omega$ 는  $[-\pi, \pi]$ 까지)
% w_conv = linspace(-pi, pi, N_conv);
%
% % plot
% figure;
%
% subplot(2, 1, 1);
% plot(w_conv, X_L_shifted);
% xlabel('\omega (rad/sample)');
% ylabel('Log Scale Magnitude');
% title('|X_L(\omega)| of x_l[n] = h_L[n] * x_2[n]');
% grid on;
%
% subplot(2, 1, 2);
% plot(w_conv, X_H_shifted);
% xlabel('\omega (rad/sample)');
% ylabel('Log Scale Magnitude');
% title('|X_H(\omega)| of x_h[n] = h_H[n] * x_2[n]');
% grid on;

```

```

%=====

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% y[n] = sum{x[m] * h[n - m]} (m=0 ~ Nx+Nh-1까지)
% discrete한 신호 convolution 시 신호의 길이 = input 2개 신호의 길이 합 - 1
function y = self_conv(x, h)
    Nx = length(x);
    Nh = length(h);
    Ny = Nx + Nh - 1;
    y = zeros(1, Ny);

    % input 2개의 신호의 x[m] * h[n - m]가 y[n]에 accumulate됨
    % 0 < n-m+1 < Nx일 때가, 두 신호(x[m], h[n - m])가 겹치는 구간
    % 겹치는 구간의 곱을 계산하여 y[n]에 합해줌
    for n = 1:Ny
        for m = 1:Nh
            if (n - m + 1 > 0) && (n - m + 1 <= Nx)
                y(n) = y(n) + x(n - m + 1) * h(m);
            end
        end
    end
end
end

```

```

% =====
% 2-2. conv() 함수 사용
% =====

clear;
clc;
close all;

%%%%%%%%%% x2.wav 및 LPF, HPF 읽기 %%%%%%%%%%
[x2, Fs] = audioread('x2.wav'); % x2[n], Fs = 44100 Hz
h1 = importdata('LPF.txt'); % LPF impulse response h_l[n]
hh = importdata('HPF.txt'); % HPF impulse response h_h[n]

%%%%%%%%%% conv() 함수로 convolution %%%%%%%%%%
x_l_conv = conv(x2, h1); % x_l[n] = h_l[n] * x2[n]
x_h_conv = conv(x2, hh); % x_h[n] = h_h[n] * x2[n]

%%%%%%%%%% n축의 범위를 정해줌 %%%%%%%%%%
n_l = 0:length(x_l_conv)-1;
n_h = 0:length(x_h_conv)-1;

%%%%%%%%%% 각각의 filter의 n-domain에서 x2[n]*h[n]을 plot %%%%%%%%%%
figure;

subplot(2, 1, 1);
plot(n_l, x_l_conv);
xlabel('n');
ylabel('Amplitude');
title('x_l[n] = h_l[n] * x_2[n] (conv)');
grid on;

subplot(2, 1, 2);
plot(n_h, x_h_conv);
xlabel('n');
ylabel('Amplitude');
title('x_h[n] = h_h[n] * x_2[n] (conv)');
grid on;

```

```

% =====
% 2-3. 필터링 결과 청취 및 비교
% =====

clear;
clc;
close all;

%%%%%%%%%% x2.wav 및 LPF, HPF 읽기 %%%%%%%%%%
[x2, Fs] = audioread('x2.wav'); % 원본 음성, Fs = 44100 Hz
h1 = importdata('LPF.txt'); % LPF
hh = importdata('HPF.txt'); % HPF

%%%%%%%%%% conv() 함수로 convolution %%%%%%%%%%
x_l = conv(x2, h1); % Low-pass filtered signal
x_h = conv(x2, hh); % High-pass filtered signal

%%%%%%%%%% sound 재생 %%%%%%%%%%
disp('x_2[n]');
sound(x2, Fs);
pause(length(x2)/Fs + 1);

disp('x_l[n]');
sound(x_l, Fs);
pause(length(x_l)/Fs + 1);

disp('x_h[n]');
sound(x_h, Fs);
pause(length(x_h)/Fs + 1);

% % =====
% % 직접 구현한 convolution 사용
%
% clear;
% clc;
% close all;
%
% %%%%%%%%%%% x2.wav 및 LPF, HPF 읽기 %%%%%%%%%%
% [x2, Fs] = audioread('x2.wav'); % 원본 음성, Fs = 44100 Hz
% h1 = importdata('LPF.txt'); % LPF
% hh = importdata('HPF.txt'); % HPF
%
% %%%%%%%%%%% self_conv() 사용 %%%%%%%%%%
% x_l_custom = self_conv(x2, h1); % Low-pass 필터링 (직접 구현한 컨볼루션)
% x_h_custom = self_conv(x2, hh); % High-pass 필터링 (직접 구현한 컨볼루션)
%
% %%%%%%%%%%% sound 재생 %%%%%%%%%%
% disp('x_2[n]');
% sound(x2, Fs);
% pause(length(x2)/Fs + 1);
%
% disp('x_l[n]');
% sound(x_l_custom, Fs);
% pause(length(x_l_custom)/Fs + 1);
%
% disp('x_h[n]');
% sound(x_h_custom, Fs);
% pause(length(x_h_custom)/Fs + 1);
%
% %%%%%%%%%%% Convolution 직접 구현한 함수 %%%%%%%%%%
% % y[n] = sum{x[m] * h[n - m]} (m=0 ~ Nx+Nh-1까지)
%
% % discrete한 신호 convolution 시 신호의 길이 = input 2개 신호의 길이 합 - 1
% function y = self_conv(x, h)
%     Nx = length(x);
%     Nh = length(h);
%     Ny = Nx + Nh - 1;
%     y = zeros(1, Ny);
%
% % input 2개의 신호의 x[m] * h[n - m]가 y[n]에 accumulate됨
% % 0 < n-m+1 < Nx일 때가, 두 신호(x[m], h[n - m])가 겹치는 구간

```

```

%      % 겹치는 구간의 곱을 계산하여 y[n]에 합해줌
%      for n = 1:Ny
%          for m = 1:Nh
%              if (n - m + 1 > 0) && (n - m + 1 <= Nx)
%                  y(n) = y(n) + x(n - m + 1) * h(m);
%              end
%          end
%      end
% end
% % =====

```

```

% =====
% 2-1 직접 구현 vs 2-2 conv 결과 비교
% =====

clear;
clc;

%%%%% x2.wav 및 LPF, HPF 읽기 %%%%%
[x2, Fs] = audioread('x2.wav');
h1 = importdata('LPF.txt');
hh = importdata('HPF.txt');

%%%%% 결과 계산 %%%%%
x_l_custom = self_conv(x2', h1');
x_l_conv = conv(x2, h1);

x_h_custom = self_conv(x2', hh');
x_h_conv = conv(x2, hh);

%%%%% 길이 맞추기, 최소값으로 맞춤 %%%%%
% 최댓값으로 맞추면, 두 신호의 길이가 안 맞을 때, 오차가 매우 커질 수 있음
min_len_l = min(length(x_l_custom), length(x_l_conv));
min_len_h = min(length(x_h_custom), length(x_h_conv));

diff_l = x_l_custom(1:min_len_l) - x_l_conv(1:min_len_l)';
diff_h = x_h_custom(1:min_len_h) - x_h_conv(1:min_len_h)';

%%%%% 오차 측정 %%%%%
fprintf('=== Low-pass filtering 차이 ===\n');
fprintf('평균 오차: %.6e\n', mean(abs(diff_l))); % 평균 절대 오차
fprintf('최대 오차: %.6e\n', max(abs(diff_l)));

fprintf('=== High-pass filtering 차이 ===\n');
fprintf('평균 오차: %.6e\n', mean(abs(diff_h))); % 평균 절대 오차
fprintf('최대 오차: %.6e\n', max(abs(diff_h)));

%%%%% Low-pass filtering 차이 plot %%%%%
figure;
subplot(2, 1, 1);
plot(0:min_len_l-1, diff_l);
xlabel('n');
ylabel('Difference');
title('Difference: x_l[n] (self\_conv - conv)');
grid on;

%%%%% High-pass filtering 차이 시각화 %%%%%
subplot(2, 1, 2);
plot(0:min_len_h-1, diff_h);
xlabel('n');
ylabel('Difference');
title('Difference: x_h[n] (self\_conv - conv)');
grid on;

%%%%% Convolution 직접 구현한 함수 %%%%%
function y = self_conv(x, h)
    Nx = length(x);
    Nh = length(h);
    Ny = Nx + Nh - 1;
    y = zeros(1, Ny);
    for n = 1:Ny
        for m = 1:Nh
            if (n - m + 1 > 0) && (n - m + 1 <= Nx)
                y(n) = y(n) + x(n - m + 1) * h(m);
            end
        end
    end
end

```